WEMAKEDEVS × FALKORDB GRAPH HACKS (OCT 15–18, 2026) AGENTIC IDE BUILD SPECIFICATION

BLASTRADIUS: Autonomous Codebase, PR Triage & SecOps Graph Agent

Multi-Tenant FalkorDB Context Engine combining Native Graph Algorithms, Hybrid Vector+FullText Indexing, Temporal Episodic Memory, and MCP Tool Execution with WCAG 2.2 AA Accessibility.

TEAM & CODE	LEAD SOLO ARCHITECT	TARGET TRACKS	FULL FALKORDB STACK
Zero Degree (HBQZVK)	**Pratham Dey (@prothamd)**	**Track 01 + Track 02 + Track 03**	**@falkordb/mcpserver + Code-Graph + QueryWeaver + Graphiti MCP**

DIRECT INSTRUCTION FOR AGENTIC IDE AI (CURSOR / WINDSURF / CLAUDE CODE)

You are building **BlastRadius** for a single full-stack developer (Pratham Dey) as a self-contained monorepo that passes the hackathon's core rule: *"If the product still works once FalkorDB is taken out, the graph is decoration."* You MUST build directly on top of FalkorDB's official open-source suite from `docs.falkordb.com/genai-tools`: **falkordb-code-graph (`cgraph`)** for AST parsing, **`@falkordb/mcpserver`** (`:8080`), **`falkordb/graphiti-knowledge-graph-mcp`** (`:8000`), **`falkordb/queryweaver`** (`:5000/mcp`), **`GraphRAG-SDK`** (`LiteLLM` / `LiteLLMEmbedder`), all 5 native graph algorithm categories, and **WCAG 2.2 AA Accessibility**.

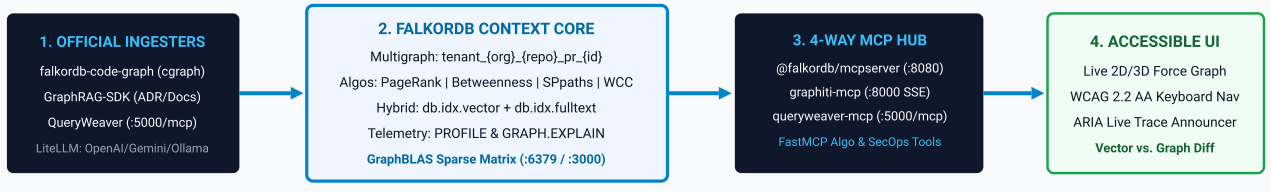
1. Multi-Track Victory Matrix & "Secret Rubric" Alignment

STRATEGY

Track	Hackathon Requirement	How BlastRadius Satisfies It Using Official Docs Tools
Track 01 (Primary) Connected Data	Multi-hop reasoning, all 5 graph algorithm types as MCP tools, explainable actions.	Combines `@falkordb/mcpserver` (`FALKORDB_DEFAULT_READONLY=true`) + custom algorithm MCP tools calling `algo.SPpaths`, `algo.pageRank`, `algo.betweenness`, `algo.WCC`, and `algo.labelPropagation` on top of `falkordb-code-graph`.
Track 02 Memory & State	Episodic, semantic, and procedural memory; persistent history; multi-agent handoffs; tenant isolation.	Runs official **`falkordb/graphiti-knowledge-graph-mcp`** (`:8000` SSE) with `group_id` tenant isolation (`add_episode`, `search`) + `:ProceduralPlaybook` nodes for `TriageAgent` → `RemediationAgent` handoffs.
Track 03 Company Brain	Knowledge from docs, tickets, SQL metrics, and chats in one graph; trace decisions to sources.	Uses **`GraphRAG-SDK`** (`kg.update()`, `apply_changes()`) for ADRs/Tickets + **`falkordb/queryweaver` MCP** (`http://127.0.0.1:5000/mcp`) for SQL CI/CD build/incident tables + `cgraph` commit ownership.

2. System Architecture & Data Flow

ARCHITECTURE



3. Official `docs.falkordb.com/genai-tools` & MCP Suite

VERIFIED DOCS STACK

To prove 100% mastery of the FalkorDB documentation to the sponsor judges, your AI IDE must wire up the exact official packages, Docker images, and LiteLLM model providers documented on `docs.falkordb.com/genai-tools`:

Official FalkorDB Tool	Exact Command / Package / Endpoint (From Docs)	How BlastRadius Uses It in Production
1. Official FalkorDB MCP Server	`npx -y @falkordb/mcpserver@latest` `FALKORDB_DEFAULT_READONLY=true` (`:8080`)	Official TypeScript MCP server exposing OpenCypher execution, graph listing, and read-only replica protection (`GRAPH.RO_QUERY`).
2. FalkorDB Code-Graph (`cgraph`)	`pip install falkordb-code-graph` `/api/find_paths`, `/api/switch_commit`	Instead of reinventing AST parsing from scratch, uses FalkorDB's own `cgraph` engine as the base layer and extends it with CVE & PageRank security triage.
3. Graphiti Memory MCP Server	`docker run -p 8000:8000 falkordb/graphiti-knowledge-graph-mcp`	Exposes SSE MCP tools (`add_episode`, `search`, `group_id` tenant isolation) for Track 02 bi-temporal episodic and semantic memory.
4. QueryWeaver Text2SQL MCP	`docker run -p 5000:5000 falkordb/queryweaver` Endpoint: `http://127.0.0.1:5000/mcp`	Exposes `list_databases`, `connect_database`, `database_schema`, and `query_database` with `MEMORY_TTL_SECONDS` to correlate SQL incidents with commits.
5. GraphRAG-SDK + LiteLLM Models	`LiteLLM('openai/gpt-4o')`, `gemini/gemini-2.5-flash`, `ollama/llama3`	Uses incremental `kg.update()`, `delete_document()`, and `apply_changes()` in CI pipelines to sync ADRs (`ADR-*.md`) with multi-model LiteLLM support.
6. All 5 Native Graph Algorithms	`algo.SPpaths`, `algo.pageRank`, `algo.betweenness`, `algo.WCC`, `algo.labelPropagation`	Wrapped in `FastMCP` (`:8090`) to give agents deterministic 1-call tools for reachability, blast radius, dead code, and microservice boundaries.
7. Multigraph Cloning & Hybrid Search	`GRAPH.COPY` + `db.idx.vector` + `db.idx.fulltext`	Clones `tenant_repo_main` to `tenant_repo_pr_142` (`GRAPH.COPY`) and combines HNSW vector embeddings with BM25 full-text CVE search.
8. AG2 / LangGraph / LlamaIndex	`FalkorGraphQueryEngine` / `FalkorDBGraphStore` + Port `:3000` Browser	Coordinates `TriageAgent` and `RemediationAgent` over the 4 MCP endpoints with one-click deep links into FalkorDB Browser (`localhost:3000`).

4. Complete Unified Graph Schema & Index Bootstrap

DATA MODEL

```
// RUN ON STARTUP TO INITIALIZE RANGE, VECTOR, AND FULL-TEXT INDEXES IN FALKORDB
CREATE INDEX FOR (f:Function) ON (f.id);
CREATE INDEX FOR (ep:API_Endpoint) ON (ep.route);
CREATE INDEX FOR (pkg:Package) ON (pkg.name);
CREATE INDEX FOR (c:CVE) ON (c.cve_id);
CREATE INDEX FOR (d:Developer) ON (d.username);
CREATE INDEX FOR (e:AgentEpisode) ON (e.session_id);

// Native HNSW Vector Index (384-dim or 1536-dim) & Native Full-Text Index
CALL db.idx.vector.createNodeIndex('Function', 'embedding', 384, 'COSINE');
CALL db.idx.vector.createNodeIndex('ArchitectureDoc', 'embedding', 384, 'COSINE');
CALL db.idx.fulltext.createNodeIndex('PastIncident', 'root_cause', 'resolution_patch');
CALL db.idx.fulltext.createNodeIndex('CVE', 'cve_id', 'description', 'vulnerable_symbol');

// Complete Multi-Track Ontology (Track 01 Code + Track 02 Memory + Track 03 Company Brain)
// (:API_Endpoint {route, method, is_public: true})-[:ROUTES_TO {weight: 1}]->(:Function)
// (:Function {id, name, community_id, pagerank_score, betweenness_score})-[:CALLS {weight: 1}]->
(:Function)
```

5. All 5 Native Algorithm Categories & Track 01 Queries

CYPHER TOOLS

```
// TOOL 1: PageRank & Betweenness Centrality (2 of the 5 Official Hackathon Algorithm Categories)
CALL algo.pageRank('Function', 'CALLS') YIELD node, score SET node.pagerank_score = score;
CALL algo.betweenness({nodeLabels: ['Function'], relationshipTypes: ['CALLS']}) YIELD node, score
SET node.betweenness_score = score
RETURN node.id AS fn_id, node.pagerank_score AS pagerank, score AS betweenness
ORDER BY betweenness DESC, pagerank DESC LIMIT 15;

// TOOL 2: Native Shortest Path — Live CVE Exploit Reachability Proof (3rd Algorithm Category)
MATCH (ep:API_Endpoint {is_public: true}),
      (pkg:Package {name: $pkg_name})-[:HAS_VULNERABILITY]->(cve:CVE {cve_id: $cve_id})
CALL algo.SPpaths({
  sourceNode: ep, targetNode: pkg,
  relTypes: ['ROUTES_TO', 'CALLS', 'USES_PACKAGE'],
  weightProp: 'weight', relDirection: 'outgoing', pathCount: 3
}) YIELD path, pathWeight
RETURN ep.route AS exposed_api, cve.cve_id AS cve, pathWeight AS hop_count,
       [n IN nodes(path) | coalesce(n.route, n.id, n.name)] AS exploit_chain
ORDER BY hop_count ASC;

// TOOL 3: Weakly Connected Components & Community Detection (4th & 5th Algorithm Categories!)
CALL algo.WCC({nodeLabels: ['Function'], relationshipTypes: ['CALLS']}) YIELD node, componentId
SET node.wcc_id = componentId;

CALL algo.labelPropagation({nodeLabels: ['Function'], relationshipTypes: ['CALLS']}) YIELD node,
communityId
SET node.community_id = communityId
RETURN communityId, count(node) AS cluster_size, collect(node.name)[0..5] AS cluster_members
ORDER BY cluster_size DESC;

// TOOL 4: Multi-Hop PR Blast Radius & Architecture Decision Record (ADR) Verification
MATCH (target:Function {id: $modified_fn_id})
OPTIONAL MATCH path = (caller:Function)-[:CALLS*1..4]->(target)
OPTIONAL MATCH (ep:API_Endpoint)-[:ROUTES_TO]->(caller)
OPTIONAL MATCH (adr:ArchitectureDoc)-[:GOVERNS]->(target)
RETURN target.id AS modified_fn, target.pagerank_score AS pr_score, target.community_id AS
cluster,
       collect(DISTINCT ep.route) AS impacted_apis,
       collect(DISTINCT {id: adr.doc_id, rule: adr.rule_summary}) AS adr_constraints;
```

6. Hybrid Search, Episodic/Procedural Memory & Company Brain

TRACK 02 & 03

```
// TOOL 5: Hybrid Vector + Full-Text + 2-Hop Structural Traversal (Single-Pass GraphRAG)
CALL db.idx.vector.queryNodes('Function', 'embedding', 5, vecf32($query_embedding)) YIELD node AS
fn, score
MATCH (fn)<-[:CALLS*0..2]-(upstream:Function)
OPTIONAL MATCH (adr:ArchitectureDoc)-[:GOVERNS]->(fn)
RETURN fn.id AS matched_fn, (1.0 - score) AS semantic_score,
       collect(DISTINCT upstream.id) AS upstream_callers, collect(DISTINCT adr.title) AS
governing_docs;

// TOOL 6: Company Brain Ownership & Hackathon Landing Page Query (Track 03)
MATCH (target:Function {id: $modified_fn_id})-[:CALLS*0..2]-(neighbor:Function)
MATCH (p:Developer)-[:AUTHORED]->(c:Commit)-[m:MODIFIED]->(neighbor)
WHERE m.invalidated_at IS NULL
RETURN p.username AS engineer, p.team AS team, count(DISTINCT c) AS commits,
       sum(m.lines_changed) AS lines_owned, collect(DISTINCT neighbor.name)[0..5] AS owned_
subgraph
ORDER BY commits DESC, lines_owned DESC LIMIT 3;

// TOOL 7: Tri-Memory Lookup — Episodic (Past Incidents), Semantic (ADRs), Procedural (Runbooks)
(Track 02)
CALL db.idx.fulltext.queryNodes('PastIncident', $error_keyword) YIELD node AS inc
MATCH (inc)-[:CAUSED_BY]->(fn:Function)
OPTIONAL MATCH (ep:AgentEpisode)-[:INVESTIGATED]->(fn)
OPTIONAL MATCH (pb:ProceduralPlaybook)-[:REMEDIATES]->(fn)
```

7. Backend Blueprint: FastMCP + Telemetry + Multigraph Sandbox

BACKEND CODE

Create `backend/mcp_server.py`. Expose all 8 tools via **FastMCP**, enforce `ro_query` for safe reads, and return `GRAPH.EXPLAIN` + `PROFILE` execution telemetry on every response:

```
from falkordb import FalkorDB
from mcp.server.fastmcp import FastMCP
```

7. Official Multi-MCP Configuration & Custom Algorithm Server

MCP ARCHITECTURE

Wire `mcp_config.json` to combine FalkorDB's 3 official MCP servers (`@falkordb/mcpserver`, `graphiti-knowledge-graph-mcp`, `queryweaver`) with your custom `blastradius-algos` FastMCP server:

```
// 1. Official mcp_config.json (Matches docs.falkordb.com/genai-
tools/mcpserver & queryweaver)
{
  "mcpServers": {
    "falkordb-official": {
      "command": "npx", "args": ["-y", "@falkordb/mcpserver@latest"],
      "env": { "FALKORDB_HOST": "localhost", "FALKORDB_PORT": "6379",
"FALKORDB_DEFAULT_READONLY": "true" }
    },
    "falkordb-graphiti-memory": { "url": "http://127.0.0.1:8000/sse" },
    "falkordb-queryweaver-sql": { "url": "http://127.0.0.1:5000/mcp" },
    "blastradius-algos": { "command": "python", "args":
["backend/mcp_server.py"] }
  }
}

# 2. backend/mcp_server.py — Custom FastMCP Server for the 5 Native
Algorithms & PR Sandbox Cloning
from falkordb import FalkorDB
from mcp.server.fastmcp import FastMCP
import time

mcp = FastMCP("BlastRadius-FalkorDB-Algos")
db = FalkorDB(host="localhost", port=6379)

def execute_instrumented(graph_name: str, cypher: str, params: dict =
None, ro: bool = True):
    g = db.select_graph(graph_name)
    t0 = time.perf_counter()
    res = g.ro_query(cypher, params) if ro else g.query(cypher, params)
    wall_ms = round((time.perf_counter() - t0) * 1000, 2)
    return {"rows": res.result_set, "engine_ms": getattr(res,
"run_time_ms", wall_ms),
```